МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу «Операционные системы»

Выполнил: В. А. Яшнов
Группа: М8О-203Б-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: приобретение практических навыков управления потоками операционной системы и обеспечения синхронизации между потоками при параллельной обработке данных.

Задание: составить программу на языке Си, обрабатывающую данные в многопоточном режиме с использованием штатных средств создания потоков ОС. Ограничение максимального количества потоков, работающих в один момент времени, задаётся ключом запуска программы.

Вариант 11: наложить K раз медианный фильтр на матрицу, состоящую из целых чисел. Размер окна задаётся пользователем.

Требования к решению:

- максимальное число одновременно работающих вычислительных потоков задаётся ключом запуска `-threads P` (лимит общий, включая главный поток);
- уметь продемонстрировать количество потоков, используемых программой, штатными средствами ОС;
- в отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков с объяснением результатов.

Реализация: Linux/POSIX — `pthread_create`, `pthread_join`, `pthread_mutex_*`, `pthread_barrier_t`, `clock_gettime(CLOCK_MONOTONIC)`; сборка — CMake (на Windows/macOS конфигурация прерывается с ошибкой).

Метод решения

Логика программы разделена на независимые модули:

1. *вычислительное ядро* `src/matrix_filter.h/.c` — медиана одного окна, фильтрация пикселя, блока строк и всей матрицы; сортировка окна элементов — быстрая сортировка `src/qsort.h/.c`;
2. *ввод/вывод* `src/matrix_io.h/.c` — матрица из файла, с клавиатуры или генерация; `src/util.h/.c` — `die()` и строгий разбор аргументов;
3. *пул потоков* `src/pool.h/.c` — распределение блоков между потоками, синхронизация итераций и измерение времени;
4. *измерения* `src/bench.h/.c` — таймер и статистика повторов (минимум, медиана);
5. *прикладной уровень* `src/app/main.c` — разбор ключей, одиночный прогон, режим `-bench`, проверка результата.

Алгоритм параллельного прогона (`filter_run`):

1. матрица режется на непересекающиеся блоки из целых строк; по умолчанию `block_rows = rows / (4P)`, так что число блоков $\approx 4P$ и работа равномерно дробится между потоками;

2. создаётся не более $P-1$ рабочих потоков: лимит `-threads P` действует вместе с главным потоком; при $P=1$ считает сам `main` без барьеров и мьютексов;

3. все потоки, включая главный, работают по одному циклу: захват номера блока из общей очереди под мьютексом, применение медианного фильтра к строкам блока *без* мьютекса (блоки не пересекаются — записей в общие данные нет), затем барьер итерации и переключение локальных буферов `a` $\leftrightarrow$ `b`;

4. барьер `pthread_barrier_t` на `P` участников — ровно один на итерацию: пока все блоки текущей итерации не обработаны, никто не читает результат предыдущей. Двойной буфер нужен, чтобы фильтрация шла по данным предыдущей итерации, а не уже перекрашенным текущей; исходная матрица `src` не изменяется;

5. счётчик взятых блоков `next_block` не сбрасывается, а растёт сквозным: на итерации `iter` доступны блоки с номерами $[\texttt{iter}\cdot T,\ (\texttt{iter}+1)\cdot T)$, где T — общее число блоков. Поэтому очередь одна на весь прогон, а критическая секция — только захват номера блока;

6. время измеряется `clock_gettime(CLOCK_MONOTONIC)` от начала создания потоков до конца последней итерации: `join` и копирование результата в измеряемый интервал не входят, создание потоков — входит.

Общий ресурс, к которому обращаются несколько потоков, — очередь блоков (`next_block` под `pthread_mutex`). Итоговые данные детерминированы: обработка блока не зависит от разбиения и числа потоков, поэтому параллельный результат можно сверить с последовательным эталоном `filter_run_serial` (ключ `-check`, вне измеряемого интервала).

Описание программы

Разделение по файлам:

`src/matrix_filter.h/.c`	медианное окно, фильтр блока строк и всей матрицы
`src/qsort.h/.c`	быстрая сортировка окна
`src/matrix_io.h/.c`	ввод из файла / с клавиатуры / генерация, вывод в файл
`src/util.h/.c`	`die()`, разбор аргументов командной строки
`src/pool.h/.c`	пул потоков: очередь, мьютекс, барьеры, таймер
`src/bench.h/.c`	таймер и статистика повторов
`src/app/main.c`	ключи запуска, одиночный прогон, `-bench`, `-check`

Основной тип данных — `filter_job`: исходная и результирующая матрицы, двойной рабочий буфер `a`/`b`, размеры `rows`/`cols`/`window`/`iters`, разбиение `block_rows`/`total_blocks`, мьютекс и барьер, сквозной счётчик `next_block`, измеренное время `elapsed`.

Функции пула потоков и их семантика:

`filter_job_init()`	инициализация задачи, создание мьютекса и буферов
`filter_job_set_block_rows()`	установка размера блока и пересчёт числа блоков
`filter_run()`	параллельный прогон на `threads` потоках
`filter_run_serial()`	последовательный эталон
`bench_stats()`	минимум и медиана по повторам

Используемые примитивы POSIX: `pthread_create`, `pthread_join`, `pthread_mutex_init/pthread_barrier_init/wait/destroy`, `clock_gettime(CLOCK_MONOTONIC)`.

Обработка ошибок: функции ввода возвращают `void` — при любой ошибке (файл не открылся, чисел не хватило, вместо числа мусор) печатается сообщение и вызывается `exit(EXIT_FAILURE)`; неудача `pthread_create`/`pthread_barrier_init` также приводит к аварийному завершению через `die()`. Ключи проверяются строго (нечётность окна, ровно один источник данных, несовместимые комбинации).

Результаты

Программа собрана и запущена под Linux (WSL2, Ubuntu). Замеры выполнялись на матрице 2048×2048, окно 3, $K = 10$ применений фильтра: `./lab3 -random -rows 2048 -cols 2048 -iters 10 -bench 16 -repeat 5 -check`. Для каждого $P \in \{1, 2, 4, 8, 16\}$ выполнялось 5 повторов, в таблице — лучшее время; $S(P) = T_1/T_P$, $E(P) = 100 \cdot S(P)/P$.

P	блоков	T_P best, мс	T_P med, мс	$S(P)$	$E(P)$, %
1	4	9013.429	9457.031	1.000	100.00
2	8	6739.713	8158.216	1.337	66.87
4	16	3551.429	3770.643	2.538	63.45
8	32	1828.926	2064.036	4.928	61.60
16	64	1095.323	1110.586	8.229	51.43

Проверка параллельного результата последовательным эталоном во всех прогонах: `check: OK`.

Объяснение результатов. До $P = 8$ ускорение растёт почти линейно ($S(8) \approx 4.9$): блоки не пересекаются, число блоков кратно числу потоков, и на синхронизацию уходит малая доля времени — критическая секция содержит только захват номера блока. Эффективность $E \approx 63\%$ ниже 100% из-за:

- барьера на каждой итерации — самый быстрый поток ждёт самый медленный; балансировка блоками по строкам не идеальна (краевой блок короче);
- алгоритм память-зависимый: окно 3×3 (сортировка 9 элементов) даёт низкое отношение вычислений к работе с памятью, потоки конкурируют за пропускную способность памяти и кэш;
- накладных расходов на создание потоков и удержание мьютекса.

При переходе от 8 к 16 потокам прирост замедляется ($S(16) \approx 8.2$, $E \approx 51\%$): исполняемых аппаратно контекстов меньше 16, поэтому часть 16 потоков фактически делит физические ядра, а издержки синхронизации (барьер с 16 участниками) растут.

Демонстрация числа потоков штатными средствами ОС. Запуск `-threads 6` на большой матрице (4096×4096, $K = 20$): `ps -eLf` показывает 6 системных потоков (LWP) одного процесса, `/proc/<pid>/status` — `Threads: 6`, `ps -L` — `NLWP 6`: главный поток + 5 созданных `pthread_create`, то есть лимит ключа запуска соблюдается.

```
$ ./lab3 --random --rows 4096 --cols 4096 --iters 20 --threads 6 &
$ ps -eLf | grep "[l]ab3"
```

```
   PID LWP C NLWP COMMAND
   632 632 92 6 ./lab3 --random --rows 4096 --cols 4096 --iters 20 --
      threads 6
   632 634 88 6 ./lab3 --random --rows 4096 --cols 4096 --iters 20 --
      threads 6
   632 635 88 6 ./lab3 --random --rows 4096 --cols 4096 --iters 20 --
      threads 6
   632 636 88 6 ./lab3 --random --rows 4096 --cols 4096 --iters 20 --
      threads 6
   632 637 88 6 ./lab3 --random --rows 4096 --cols 4096 --iters 20 --
      threads 6
   632 638 88 6 ./lab3 --random --rows 4096 --cols 4096 --iters 20 --
      threads 6
$ grep -i Threads /proc/632/status
Threads: 6
$ ps -o pid,lwp,nlwp,comm -L -p 632
   PID LWP NLWP COMMAND
   632 632 6 lab3
   632 634 6 lab3
   632 635 6 lab3
   632 636 6 lab3
   632 637 6 lab3
   632 638 6 lab3
```

Листинг 1: ps -eLf / /proc/<pid>/status при –threads 6

Выводы

В ходе выполнения лабораторной работы изучены и использованы штатные средства работы с потоками ОС Linux: `pthread_create`, `pthread_join`, мьютексы `pthread_mutex_*` и барьеры `pthread_barrier_t`, а также измерение времени через `clock_gettime(CLOCK_MONOT`

Разработана многопоточная программа варианта 11: K-кратное наложение медианного фильтра на матрицу целых чисел с задаваемым пользовательским окном. Матрица разбивается на непересекающиеся блоки строк, которые захватываются потоками из общей очереди под мьютексом; границы итераций синхронизируются барьером, необходимым для корректного переключения двойного буфера a/b. Лимит `-threads P` ограничивает общее число вычислительных потоков вместе с главным, при $P = 1$ программа работает полностью последовательно.

Корректность подтверждена сверкой с последовательным эталоном (`-check`) при всех значениях P; число потоков продемонстрировано штатными средствами ОС (`ps -eLf`, `top -H`, `/proc/<pid>/status`). Экспериментально получены ускорение $S(P) = T_1/T_P$ и эффективность $E(P) = 100\, S(P)/P$: на 8 потоках достигается пятикратное ускорение, при дальнейшем росте P выгоды сокращаются из-за накладных расходов на синхронизацию и уменьшения доли работы на поток. Сформирован полноценный отчёт о ходе и результатах работы.

Исходная программа

Сборка и запуск:

- Linux: `cmake -S . -B build-linux && cmake -build build-linux`, запуск `./build-linux/bin/lab3` (полные варианты команд — в `run.md`);
- пример исследования: `./lab3 -random -rows 2048 -cols 2048 -iters 10 -bench 16 -repeat 5 -check`;
- пример обычного прогона: `./lab3 -manual -threads 4 -window 3 -iters 5`.

Вычислительное ядро:

```
#ifndef LAB3_MATRIX_FILTER_H
#define LAB3_MATRIX_FILTER_H

#include <stddef.h>

int median(int* data, size_t size);

void median_filter_pixel(const int* in_matrix, int* out_matrix, size_t rows
    , size_t cols,
                 size_t pixel_row, size_t pixel_col,
                 size_t window_width,
                 int* window_buffer);

void median_filter_block(const int* in_matrix, int* out_matrix, size_t rows
    , size_t cols,
                 size_t first_row, size_t last_row,
                 size_t window_width,
                 int* window_buffer);

void median_filter_apply(const int* in_matrix, int* out_matrix, size_t rows
    , size_t cols,
                 size_t window_width,
                 int* window_buffer);

size_t filter_window_buffer_size(size_t window_width);

#endif
```

Листинг 2: src/matrix_filter.h

```
#include <stddef.h>

#include "matrix_filter.h"
#include "qsort.h"

int median(int* data, size_t size) {
   int_qsort(data, size);
   return data[size/2];
}

```

```c
void median_filter_pixel(const int* in_matrix, int* out_matrix, size_t rows, size_t cols,
                         size_t pixel_row, size_t pixel_col,
                         size_t window_width,
                         int* window_buffer) {
   int half = (int)(window_width / 2);
   size_t cnt = 0;
   for (int i = -half; i <= half; ++i) {
      for (int j = -half; j <= half; ++j) {
         int r = (int)pixel_row + i;
         int c = (int)pixel_col + j;
         if (r >= 0 && c >= 0 && r < (int)rows && c < (int)cols) {
            window_buffer[cnt++] = in_matrix[(size_t)r*cols+(size_t)c];
         }
      }
   }
   out_matrix[pixel_row*cols+pixel_col] = median(window_buffer, cnt);
}

void median_filter_block(const int* in_matrix, int* out_matrix, size_t rows, size_t cols,
                         size_t first_row, size_t last_row,
                         size_t window_width,
                         int* window_buffer) {
   if (first_row >= last_row) {
      return;
   }
   for (size_t r = first_row; r < last_row; ++r) {
      for (size_t c = 0; c < cols; ++c) {
         median_filter_pixel(in_matrix, out_matrix, rows, cols, r, c,
            window_width, window_buffer);
      }
   }
}

void median_filter_apply(const int* in_matrix, int* out_matrix, size_t rows, size_t cols,
                         size_t window_width,
                         int* window_buffer) {
   median_filter_block(in_matrix, out_matrix, rows, cols, 0, rows,
      window_width, window_buffer);
}

size_t filter_window_buffer_size(size_t window_width) {
   return window_width * window_width;
}
```

Листинг 3: src/matrix_filter.c

```c
#ifndef LAB3_QSORT_H
#define LAB3_QSORT_H

#include <stddef.h>

size_t part(int *data, size_t lo, size_t hi);

void qsort_core(int *data, size_t lo, size_t hi);
```

```
9
10 void int_qsort(int *data, size_t size);
11
12 #endif
```

Листинг 4: src/qsort.h

```
1  #include <stddef.h>
2
3  #include "qsort.h"
4
5  size_t part(int* data, size_t lo, size_t hi) {
6     int pivot = data[lo + (hi - lo) / 2];
7     int i = lo-1;
8     int j = hi+1;
9     while (1) {
10       do {j--;} while (data[j] > pivot);
11       do {i++;} while (data[i] < pivot);
12       if (i >= j) {
13          return j;
14       }
15       int temp = data[j];
16       data[j] = data[i];
17       data[i] = temp;
18    }
19 }
20
21 void qsort_core(int* data, size_t lo, size_t hi) {
22    if (lo < hi) {
23       size_t p = part(data, lo, hi);
24       qsort_core(data, lo, p);
25       qsort_core(data, p+1, hi);
26    }
27
28 }
29
30 void int_qsort(int* data, size_t size) {
31    if (size < 2) {
32       return;
33    }
34    qsort_core(data, 0, size-1);
35 }
```

Листинг 5: src/qsort.c

Пул потоков и синхронизация:

```
1  #ifndef LAB3_POOL_H
2  #define LAB3_POOL_H
3
4  #include <pthread.h>
5  #include <stddef.h>
6
7  #define POOL_MAX_THREADS 1024u
```

```
8
9  typedef struct {
10     int *src; /* исходнаяматрица, неизменяется*/
11     int *dst; /* послепрогонахранитрезультат*/
12
13     int *a; /* рабочийбуфер: наитерацииодинчитается, другойпишется*/
14     int *b; /* второйрабочийбуфер*/
15
16     size_t rows;
17     size_t cols;
18     size_t window;
19     size_t iters;
20     size_t block_rows;
21     size_t total_blocks;
22
23     pthread_mutex_t mutex; /* защищаеточенедьблоковиобщиесчётчики*/
24     pthread_barrier_t barrier; /* границыитераций*/
25
26     size_t next_block; /* взятыхблоковзапрогон, сквознойсчёт*/
27     double elapsed; /* времяпрогонавсекундах, заполняетсяfilter_run */
28 } filter_job;
29
30 int filter_job_init(filter_job *job, int *src, int *dst, size_t rows,
       size_t cols,
31                     size_t window, size_t iters, size_t block_rows);
32
33 void filter_job_destroy(filter_job *job);
34
35 void filter_job_set_block_rows(filter_job *job, size_t block_rows);
36
37 int filter_run(filter_job *job, size_t threads);
38
39 int filter_run_serial(filter_job *job);
40
41 #endif
```

Листинг 6: src/pool.h

```
1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  #include "bench.h"
7  #include "matrix_filter.h"
8  #include "pool.h"
9  #include "util.h"
10
11 static int *alloc_ints(size_t n)
12 {
13     int *data = (int *)malloc(n * sizeof(*data));
14     if (data == NULL) {
15         die("out of memory");
16     }
17     return data;
18 }
19
```

```
20 static int *window_buffer_new(const filter_job *job)
21 {
22    return alloc_ints(filter_window_buffer_size(job->window));
23 }
24
25 static void run_start(filter_job *job)
26 {
27    memcpy(job->a, job->src, job->rows * job->cols * sizeof(*job->a));
28
29    pthread_mutex_lock(&job->mutex);
30    job->next_block = 0;
31    pthread_mutex_unlock(&job->mutex);
32 }
33
34 static int block_take(filter_job *job, size_t iter, size_t *block)
35 {
36    int taken = 0;
37    size_t base = iter * job->total_blocks;
38    size_t limit = base + job->total_blocks;
39
40    pthread_mutex_lock(&job->mutex);
41    if (job->next_block < limit) {
42       *block = job->next_block++ - base;
43       taken = 1;
44    }
45    pthread_mutex_unlock(&job->mutex);
46    return taken ? 0 : -1;
47 }
48
49 static void run_blocks(filter_job *job, size_t iter, const int *in, int *
       out, int *window_buffer)
50 {
51    size_t block = 0;
52
53    while (block_take(job, iter, &block) == 0) {
54       size_t from = block * job->block_rows;
55       size_t to = from + job->block_rows;
56       if (to > job->rows) {
57          to = job->rows;
58       }
59
60       median_filter_block(in, out, job->rows, job->cols, from, to,
61                           job->window, window_buffer);
62    }
63 }
64
65 static const int *run_iterations(filter_job *job, int *window_buffer)
66 {
67    int *in = job->a;
68    int *out = job->b;
69
70    for (size_t iter = 0; iter < job->iters; ++iter) {
71       if (iter > 0) {
72          int *tmp = in;
73          in = out;
74          out = tmp;
75       }
76       run_blocks(job, iter, in, out, window_buffer);
```

```
77
78          pthread_barrier_wait(&job->barrier);
79      }
80      return out;
81  }
82
83  static void *worker_main(void *arg)
84  {
85      filter_job *job = (filter_job *)arg;
86      int *window_buffer = window_buffer_new(job);
87
88      run_iterations(job, window_buffer);
89      free(window_buffer);
90      return NULL;
91  }
92
93  int filter_job_init(filter_job *job, int *src, int *dst, size_t rows,
        size_t cols,
94                  size_t window, size_t iters, size_t block_rows)
95  {
96      size_t cells = rows * cols;
97
98      memset(job, 0, sizeof(*job));
99      if (pthread_mutex_init(&job->mutex, NULL) != 0) {
100         fprintf(stderr, "[pool] pthread_mutex_init failed\n");
101         return -1;
102     }
103
104     job->src = src;
105     job->dst = dst;
106     job->rows = rows;
107     job->cols = cols;
108     job->window = window;
109     job->iters = iters;
110
111     job->a = alloc_ints(cells);
112     job->b = alloc_ints(cells);
113
114     filter_job_set_block_rows(job, block_rows);
115     return 0;
116 }
117
118 void filter_job_set_block_rows(filter_job *job, size_t block_rows)
119 {
120     if (block_rows == 0 || block_rows > job->rows) {
121         block_rows = job->rows;
122     }
123     job->block_rows = block_rows;
124     job->total_blocks = (job->rows + block_rows - 1) / block_rows;
125 }
126
127 void filter_job_destroy(filter_job *job)
128 {
129     pthread_mutex_destroy(&job->mutex);
130     free(job->a);
131     free(job->b);
132     job->a = NULL;
133     job->b = NULL;
```

```
}

static void save_result(filter_job *job, const int *result)
{
   memcpy(job->dst, result, job->rows * job->cols * sizeof(*job->dst));
}

int filter_run_serial(filter_job *job)
{
   int *window_buffer = window_buffer_new(job);
   int *in = job->a;
   int *out = job->b;

   run_start(job);

   double t0 = now_sec();
   for (size_t iter = 0; iter < job->iters; ++iter) {
      if (iter > 0) {
         int *tmp = in;
         in = out;
         out = tmp;
      }
      median_filter_apply(in, out, job->rows, job->cols, job->window,
         window_buffer);
   }
   job->elapsed = now_sec() - t0;

   save_result(job, out);
   free(window_buffer);
   return 0;
}

int filter_run(filter_job *job, size_t threads)
{
   pthread_t *tids = NULL;
   size_t workers = 0;

   if (threads == 0) {
      threads = 1;
   }
   if (threads == 1) {
      return filter_run_serial(job);
   }
   if (threads > POOL_MAX_THREADS) {
      fprintf(stderr, "[pool] too many threads: %zu (max %u)\n", threads,
         POOL_MAX_THREADS);
      return -1;
   }

   workers = threads - 1;
   tids = (pthread_t *)calloc(workers, sizeof(*tids));
   if (tids == NULL) {
      die("out of memory");
   }
   int *main_buffer = window_buffer_new(job);

   if (pthread_barrier_init(&job->barrier, NULL, threads) != 0) {
      die("cannot create barrier");
```

```
190     }
191     run_start(job);
192
193     double t0 = now_sec();
194     for (size_t i = 0; i < workers; ++i) {
195        if (pthread_create(&tids[i], NULL, worker_main, job) != 0) {
196           die("cannot create thread");
197        }
198     }
199
200     const int *result = run_iterations(job, main_buffer);
201     job->elapsed = now_sec() - t0;
202
203     save_result(job, result);
204
205     for (size_t i = 0; i < workers; ++i) {
206        pthread_join(tids[i], NULL);
207     }
208
209     pthread_barrier_destroy(&job->barrier);
210
211     free(main_buffer);
212     free(tids);
213     return 0;
214 }
```

Листинг 7: src/pool.c

Ввод/вывод и утилиты:

```
1  #ifndef LAB3_MATRIX_IO_H
2  #define LAB3_MATRIX_IO_H
3
4  #include <stddef.h>
5
6  #define MATRIX_MAX_DIM 100000u
7
8  typedef struct {
9     int *data;
10    size_t rows;
11    size_t cols;
12 } matrix;
13
14 void matrix_load_file(const char *path, matrix *m, size_t rows, size_t cols
      );
15
16 void matrix_load_stdin(matrix *m, size_t rows, size_t cols);
17
18 void matrix_generate(matrix *m, size_t rows, size_t cols, unsigned long
      seed, int min_val, int max_val);
19
20 void matrix_save(const char *path, const matrix *m);
21
22 #endif
```

Листинг 8: src/matrix_io.h

```c
#include <stdio.h>
#include <stdlib.h>

#include "matrix_io.h"
#include "util.h"

static int *matrix_alloc(matrix *m, size_t rows, size_t cols)
{
   int *data = (int *)malloc(rows * cols * sizeof(*data));

   if (data == NULL) {
      die("out of memory");
   }
   m->data = data;
   m->rows = rows;
   m->cols = cols;
   return data;
}

void matrix_load_file(const char *path, matrix *m, size_t rows, size_t cols
   )
{
   FILE *f = fopen(path, "r");
   if (f == NULL) {
      die("cannot open file");
   }

   size_t n = rows * cols;
   int *data = matrix_alloc(m, rows, cols);
   for (size_t i = 0; i < n; ++i) {
      if (fscanf(f, "%d", &data[i]) != 1) {
         die("unexpected end of file");
      }
   }

   fclose(f);
}

void matrix_load_stdin(matrix *m, size_t rows, size_t cols)
{
   if (rows == 0) {
      printf("Enter rows: ");
      fflush(stdout);
      if (scanf("%zu", &rows) != 1) {
         die("invalid rows\n");
      }
   }
   if (cols == 0) {
      printf("Enter cols: ");
      fflush(stdout);
      if (scanf("%zu", &cols) != 1) {
         die("invalid cols\n");
      }
   }

   size_t n = rows * cols;
   int *data = matrix_alloc(m, rows, cols);

```

```
58     printf("Enter %zu integers:\n> ", n);
59     fflush(stdout);
60     for (size_t i = 0; i < n; ++i) {
61        if (scanf("%d", &data[i]) != 1) {
62           die("invalid integer");
63        }
64     }
65  }
66
67  void matrix_generate(matrix *m, size_t rows, size_t cols, unsigned long
       seed, int min_val, int max_val)
68  {
69     if (min_val > max_val) {
70        die("min value must not exceed max");
71     }
72
73     int *data = matrix_alloc(m, rows, cols);
74     int span = max_val - min_val + 1;
75
76     srand((unsigned int)seed);
77     for (size_t i = 0, n = rows * cols; i < n; ++i) {
78        data[i] = min_val + rand() % span;
79     }
80  }
81
82  void matrix_save(const char *path, const matrix *m)
83  {
84     FILE *f = fopen(path, "w");
85     if (f == NULL) {
86        die("cannot create file");
87     }
88
89     for (size_t r = 0; r < m->rows; ++r) {
90        for (size_t c = 0; c < m->cols; ++c) {
91           int rc = fprintf(f, "%d%c", m->data[r * m->cols + c], c + 1 < m->
                cols ? ' ' : '\n');
92           if (rc < 0) {
93              die("write error");
94           }
95        }
96     }
97
98     if (fclose(f) != 0) {
99        die("cannot close file");
100    }
101 }
```

Листинг 9: src/matrix_io.c

```
1  #ifndef LAB3_UTIL_H
2  #define LAB3_UTIL_H
3
4  #include <stddef.h>
5
6  void die(const char *msg);
7
8  size_t parse_size_range(const char *s, size_t min, size_t max);
```

```
9
10 #endif
```

Листинг 10: src/util.h

```
1  #include <errno.h>
2  #include <stdint.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  #include "util.h"
7
8  void die(const char *msg)
9  {
10    fprintf(stderr, "%s\n", msg);
11    exit(EXIT_FAILURE);
12 }
13
14 static int all_digits(const char *s, size_t from)
15 {
16    if (s[from] == '\0') {
17       return 0;
18    }
19    for (const char *p = s + from; *p != '\0'; ++p) {
20       if (*p < '0' || *p > '9') {
21          return 0;
22       }
23    }
24    return 1;
25 }
26
27 static int parse_long(const char *s, long *out)
28 {
29    char *end = NULL;
30
31    if (s == NULL || *s == '\0') {
32       return -1;
33    }
34    if ((*s == '+' || *s == '-') && !all_digits(s, 1)) {
35       return -1;
36    }
37
38    errno = 0;
39    long value = strtol(s, &end, 10);
40    if (errno == ERANGE || end == s || end == NULL || *end != '\0') {
41       return -1;
42    }
43    *out = value;
44    return 0;
45 }
46
47 size_t parse_size_range(const char *s, size_t min, size_t max)
48 {
49    long value = 0;
50
51    if (parse_long(s, &value) != 0) {
52       die("argument is not an integer");
```

```
    }
    if (value < 0 || (unsigned long)value > (unsigned long)SIZE_MAX) {
        die("argument out of range");
    }
    if ((size_t)value < min || (size_t)value > max) {
        die("argument out of range");
    }
    return (size_t)value;
}
```

Листинг 11: src/util.c

Измерения:

```
#ifndef LAB3_BENCH_H
#define LAB3_BENCH_H

#include <stddef.h>

double now_sec(void);

void bench_stats(const double *data, size_t n, double *best, double *median
    );

#endif
```

Листинг 12: src/bench.h

```
#include <stddef.h>
#include <stdlib.h>
#include <time.h>

#include "bench.h"

double now_sec(void)
{
    struct timespec ts;

    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {
        return 0.0;
    }
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}

static int cmp_double(const void *a, const void *b)
{
    double x = *(const double *)a;
    double y = *(const double *)b;

    return (x > y) - (x < y);
}

void bench_stats(const double *data, size_t n, double *best, double *median
    )
```

```
{
   if (n == 0) {
      *best = 0.0;
      *median = 0.0;
      return;
   }

   double local_min = data[0];
   for (size_t i = 1; i < n; ++i) {
      if (data[i] < local_min) {
         local_min = data[i];
      }
   }
   *best = local_min;

   double *sorted = (double *)malloc(n * sizeof(*sorted));
   if (sorted == NULL) {
      *median = *best;
      return;
   }
   for (size_t i = 0; i < n; ++i) {
      sorted[i] = data[i];
   }
   qsort(sorted, n, sizeof(*sorted), cmp_double);
   *median = (n % 2 == 1) ? sorted[n / 2] : 0.5 * (sorted[n / 2 - 1] +
       sorted[n / 2]);
   free(sorted);
}
```

Листинг 13: src/bench.c

Прикладной уровень и файл сборки:

```
#include <getopt.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "bench.h"
#include "matrix_io.h"
#include "pool.h"
#include "util.h"

#define MAX_ITERS 10000u
#define MAX_WINDOW 999u
#define MAX_REPEAT 1000u
#define BLOCKS_PER_THREAD 4u
#define GEN_MIN 0
#define GEN_MAX 255

enum {
   OPT_ROWS = 256,
   OPT_COLS,
   OPT_SEED,
   OPT_REPEAT,
```

```
    OPT_BENCH,
    OPT_SAVE
};

typedef struct {
    size_t threads;
    size_t window;
    size_t iters;
    size_t repeat;
    size_t block;
    size_t rows;
    size_t cols;
    size_t bench_max;
    unsigned long seed;
    const char *input;
    const char *save;
    int manual;
    int random_src;
    int check;
    int print_matrix;
    int threads_given;
} options;

static void usage(FILE *out, const char *prog)
{
    fprintf(out,
        "usage: %s --threads P (--input FILE | --manual | --random) [options]\n"
        "\n"
        " -t, --threads P максимумодновременноработающихвычислительныхпотоков,\n"
        " включаяглавный(1..%u, поумолчанию1)\n"
        " -w, --window W ширинаокнафильтра, нечётноечислопо умолчанию3)\n"
        " -k, --iters K числоприменениймедианногофильтрапо умолчанию1)\n"
        " -i, --input FILE матрицаизфайла: ровноrows*cols целыхчисел\n"
        " -m, --manual вводматрицысклавиатуры\n"
        " -r, --random сгенерироватьматрицунужны --rows и--cols)\n"
        " --rows R числострок\n"
        " --cols C числостолбцов\n"
        " --seed S начальноезначениегенераторапо умолчанию1)\n"
        " -b, --block N строкводномблокезадачипо умолчаниюrows/(4*P))\n"
        " --repeat N повторовизмеренияпо умолчанию1)\n"
        " -c, --check сверитьрезультатспоследовательнымэталоном\n"
        " --bench P прогнатьP = 1, 2, 4, ... ивывестиT_p, S(P), E(P)\n"
        " --save FILE сохранитьрезультатвфайл\n"
        " -p, --print-matrix вывестирезультирующуюматрицу\n"
        " -h, --help этасправка\n",
        prog, POOL_MAX_THREADS);
}

static size_t auto_block_rows(size_t rows, size_t threads)
{
    size_t block = rows / (threads * BLOCKS_PER_THREAD);

    if (block == 0) {
        block = 1;
    }
    if (block > rows) {
```

```
        block = rows;
    }
    return block;
}

static int compare_with_reference(const int *data, const int *ref, size_t
    rows, size_t cols)
{
    for (size_t i = 0, n = rows * cols; i < n; ++i) {
        if (data[i] != ref[i]) {
            fprintf(stderr, "check FAILED at row %zu col %zu: got %d, expected
                %d\n",
                  i / cols, i % cols, data[i], ref[i]);
            return -1;
        }
    }
    printf("check: OK\n");
    return 0;
}

static void print_matrix(const char *title, const int *data, size_t rows,
    size_t cols)
{
    printf("%s (%zux%zu)\n", title, rows, cols);
    for (size_t r = 0; r < rows; ++r) {
        printf("%4zu:", r);
        for (size_t c = 0; c < cols; ++c) {
            printf(" %d", data[r * cols + c]);
        }
        printf("\n");
    }
}

int main(int argc, char **argv)
{
    static const struct option long_options[] = {
        {"threads", required_argument, NULL, 't'},
        {"window", required_argument, NULL, 'w'},
        {"iters", required_argument, NULL, 'k'},
        {"input", required_argument, NULL, 'i'},
        {"manual", no_argument, NULL, 'm'},
        {"random", no_argument, NULL, 'r'},
        {"block", required_argument, NULL, 'b'},
        {"check", no_argument, NULL, 'c'},
        {"print-matrix", no_argument, NULL, 'p'},
        {"help", no_argument, NULL, 'h'},
        {"rows", required_argument, NULL, OPT_ROWS},
        {"cols", required_argument, NULL, OPT_COLS},
        {"seed", required_argument, NULL, OPT_SEED},
        {"repeat", required_argument, NULL, OPT_REPEAT},
        {"bench", required_argument, NULL, OPT_BENCH},
        {"save", required_argument, NULL, OPT_SAVE},
        {NULL, 0, NULL, 0}
    };

    options cfg;
    memset(&cfg, 0, sizeof(cfg));
    cfg.threads = 1;
```

```
135     cfg.window = 3;
136     cfg.iters = 1;
137     cfg.repeat = 1;
138     cfg.seed = 1;
139
140     int opt;
141     while ((opt = getopt_long(argc, argv, "t:w:k:i:mrb:chp", long_options,
            NULL)) != -1) {
142         switch (opt) {
143         case 't':
144             cfg.threads = parse_size_range(optarg, 1, POOL_MAX_THREADS);
145             cfg.threads_given = 1;
146             break;
147         case 'w':
148             cfg.window = parse_size_range(optarg, 1, MAX_WINDOW);
149             break;
150         case 'k':
151             cfg.iters = parse_size_range(optarg, 1, MAX_ITERS);
152             break;
153         case 'i':
154             cfg.input = optarg;
155             break;
156         case 'm':
157             cfg.manual = 1;
158             break;
159         case 'r':
160             cfg.random_src = 1;
161             break;
162         case 'b':
163             cfg.block = parse_size_range(optarg, 1, MATRIX_MAX_DIM);
164             break;
165         case 'c':
166             cfg.check = 1;
167             break;
168         case 'p':
169             cfg.print_matrix = 1;
170             break;
171         case 'h':
172             usage(stdout, argv[0]);
173             return 0;
174         case OPT_ROWS:
175             cfg.rows = parse_size_range(optarg, 1, MATRIX_MAX_DIM);
176             break;
177         case OPT_COLS:
178             cfg.cols = parse_size_range(optarg, 1, MATRIX_MAX_DIM);
179             break;
180         case OPT_SEED:
181             cfg.seed = (unsigned long)parse_size_range(optarg, 1, 1000000);
182             break;
183         case OPT_REPEAT:
184             cfg.repeat = parse_size_range(optarg, 1, MAX_REPEAT);
185             break;
186         case OPT_BENCH:
187             cfg.bench_max = parse_size_range(optarg, 1, POOL_MAX_THREADS);
188             break;
189         case OPT_SAVE:
190             cfg.save = optarg;
191             break;
```

```
192         default:
193             usage(stderr, argv[0]);
194             return 1;
195         }
196     }
197
198     if (optind < argc) {
199         fprintf(stderr, "unexpected argument '%s'\n", argv[optind]);
200         usage(stderr, argv[0]);
201         return 1;
202     }
203
204     int sources = (cfg.input != NULL) + cfg.manual + cfg.random_src;
205     if (sources == 0) {
206         fprintf(stderr, "no input source: use --input FILE, --manual or --
                random\n");
207         usage(stderr, argv[0]);
208         return 1;
209     }
210     if (sources > 1) {
211         die("choose exactly one input source: --input, --manual or --random")
                ;
212     }
213     if (cfg.input != NULL && (cfg.rows == 0 || cfg.cols == 0)) {
214         die("--input needs --rows and --cols: the file contains only numbers"
                );
215     }
216     if (cfg.random_src && (cfg.rows == 0 || cfg.cols == 0)) {
217         die("--random needs --rows and --cols");
218     }
219     if (cfg.window % 2 == 0) {
220         die("--window must be odd");
221     }
222     if (cfg.bench_max != 0 && cfg.threads_given) {
223         die("--bench sets the thread count itself, --threads cannot be
                combined with it");
224     }
225     if (cfg.bench_max == 0 && cfg.repeat != 1) {
226         die("--repeat only makes sense together with --bench");
227     }
228
229     matrix input = {NULL, 0, 0};
230     if (cfg.input != NULL) {
231         matrix_load_file(cfg.input, &input, cfg.rows, cfg.cols);
232     } else if (cfg.manual) {
233         matrix_load_stdin(&input, cfg.rows, cfg.cols);
234     } else {
235         matrix_generate(&input, cfg.rows, cfg.cols, cfg.seed, GEN_MIN,
                GEN_MAX);
236     }
237
238     int *src = input.data;
239     size_t rows = input.rows;
240     size_t cols = input.cols;
241     size_t cells = rows * cols;
242
243     int *dst = (int *)malloc(cells * sizeof(*dst));
244     int *ref = cfg.check ? (int *)malloc(cells * sizeof(*ref)) : NULL;
```

```
245     if (dst == NULL || (cfg.check && ref == NULL)) {
246         die("out of memory");
247     }
248
249     filter_job job;
250     if (filter_job_init(&job, src, dst, rows, cols, cfg.window, cfg.iters,
            1) != 0) {
251         die("cannot init filter job");
252     }
253
254     int rc = 0;
255
256     if (cfg.bench_max == 0) {
257         size_t block = cfg.block != 0 ? cfg.block : auto_block_rows(rows, cfg
                .threads);
258         filter_job_set_block_rows(&job, block);
259
260         if (filter_run(&job, cfg.threads) != 0) {
261             die("filter_run failed");
262         }
263         printf("elapsed=%.6f s\n", job.elapsed);
264     } else {
265         double *times = (double *)malloc(cfg.repeat * sizeof(*times));
266         if (times == NULL) {
267             die("out of memory");
268         }
269
270         printf("%-5s %-8s %-15s %-15s %-8s %-9s\n",
271                "P", "blocks", "T_p best (ms)", "T_p med (ms)", "S(P)", "E(P),%"
                    );
272         printf("-------------------------------------------------------------\n
                ");
273
274         double base = 0.0;
275         for (size_t p = 1; p <= cfg.bench_max; p *= 2) {
276             size_t block = cfg.block != 0 ? cfg.block : auto_block_rows(rows,
                    p);
277             filter_job_set_block_rows(&job, block);
278
279             for (size_t r = 0; r < cfg.repeat; ++r) {
280                 if (filter_run(&job, p) != 0) {
281                     die("filter_run failed");
282                 }
283                 times[r] = job.elapsed;
284             }
285
286             double best = 0.0;
287             double median = 0.0;
288             bench_stats(times, cfg.repeat, &best, &median);
289             if (p == 1) {
290                 base = best;
291             }
292
293             double speedup = base / best;
294             double efficiency = 100.0 * speedup / (double)p;
295
296             printf("%-5zu %-8zu %-15.3f %-15.3f %-8.3f %-9.2f\n",
297                    p, job.total_blocks, best * 1e3, median * 1e3, speedup,
```

```
                    efficiency);
        }
        free(times);
    }

    if (rc == 0 && cfg.check) {
        filter_job reference;
        if (filter_job_init(&reference, src, ref, rows, cols, cfg.window, cfg
            .iters, 1) != 0) {
            die("cannot init filter job");
        }
        if (filter_run_serial(&reference) != 0) {
            die("serial reference failed");
        }
        if (compare_with_reference(dst, ref, rows, cols) != 0) {
            rc = 2;
        }
        filter_job_destroy(&reference);
    }

    if (rc == 0 && cfg.print_matrix) {
        print_matrix("result", dst, rows, cols);
    }

    if (rc == 0 && cfg.save != NULL) {
        matrix result = {dst, rows, cols};
        matrix_save(cfg.save, &result);
    }

    filter_job_destroy(&job);
    free(src);
    free(dst);
    free(ref);
    return rc;
}
```

Листинг 14: src/app/main.c

```
cmake_minimum_required(VERSION 3.15)

project(lab3 C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)

if(NOT (UNIX AND NOT APPLE))
    message(FATAL_ERROR "lab3 is a POSIX/Linux lab (pthreads), current
        platform is not supported")
endif()

message(STATUS "Building for Linux (POSIX threads)...")

add_compile_options(-Wall -Wextra -Wpedantic)

```

```
find_package(Threads REQUIRED)

add_library(filter STATIC
   src/matrix_filter.h
   src/matrix_filter.c
   src/qsort.h
   src/qsort.c
)
target_include_directories(filter PUBLIC src)
target_compile_definitions(filter PUBLIC _POSIX_C_SOURCE=200809L)

add_library(core STATIC
   src/matrix_io.h
   src/matrix_io.c
   src/util.h
   src/util.c
   src/pool.h
   src/pool.c
   src/bench.h
   src/bench.c
)
target_link_libraries(core PUBLIC filter Threads::Threads)

add_executable(lab3 src/app/main.c)
target_link_libraries(lab3 PRIVATE core Threads::Threads)
```

Листинг 15: CMakeLists.txt